Preprocessing, Compiling, Assembly, Linking

Dal codice sorgente all'eseguibile

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

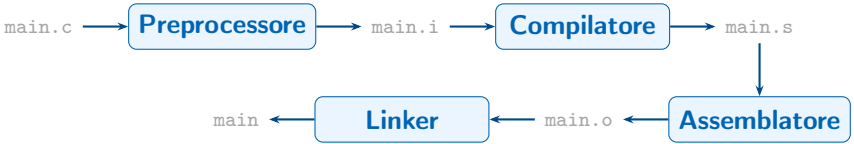
April 20, 2026

UniMe
1548

$$\texttt{main.c} \xrightarrow{\text{gcc}} \texttt{main.i} \xrightarrow{\text{cc1}} \texttt{main.s} \xrightarrow{\text{as}} \texttt{main.o} \xrightarrow{\text{ld}} \texttt{main}$$

Il processo di compilazione — Panoramica

GCC trasforma un file `.c` in un eseguibile in **4 fasi distinte**:



Flag per fermarsi a ogni fase

`-E` → preprocessing `-S` → compilation
`-c` → assembly

Flag utili

`-O0` nessuna ottimizzazione
`-masm=intel` sintassi Intel `-o nome` nome output

Fase 1 — Preprocessing

Il preprocessore `cpp` agisce *prima* del compilatore:

- Espande le macro `#define`
- Include i file header `#include`
- Gestisce le direttive condizionali `#ifdef`
- Rimuove i commenti

```
gcc -E main.c -o main.i    # produce codice C "puro" espanso
```

Attenzione

Il file `.i` puo' essere molto piu' grande del sorgente: un singolo `#include <stdio.h>` espande migliaia di righe.

Fase 2 — Compilazione

Il compilatore traduce C preprocessato in **assembly** per l'architettura target.

- Analisi sintattica e semantica
- Ottimizzazioni (`-O0`, `-O1`, `-O2`, `-O3`)
- Generazione del codice assembly

```
gcc -O0 -S -masm=intel main.c -o main.s
```

Perche' `-O0`?

Con ottimizzazioni disabilitate, l'assembly rispecchia fedelmente il codice C. Fondamentale per lo studio e il debug.

Fase 3 & 4 — Assemblaggio e Linking

Assemblatore (`as`)

- Traduce assembly → codice macchina binario
- Produce un file oggetto `.o`
- I riferimenti esterni sono ancora non risolti

```
gcc -c main.c -o main.o
```

Linker (`ld`)

- Combina `.o` e librerie
- Risolve tutti i riferimenti simbolici
- Produce l'eseguibile finale

```
gcc main.o -o main
```

Nella pratica: `gcc main.c -o main` esegue tutte e 4 le fasi

ISA vs Microarchitettura

ISA — Instruction Set Architecture

Il **contratto** tra software e hardware.

Definisce:

- Quali istruzioni esistono
- Come si codificano in binario
- Quali registri ci sono

L'assembly parla con l'ISA.

Microarchitettura

Come l'ISA e' implementata nel silicio.

- Invisibile al software
- Determina le prestazioni
- Diversa per ogni CPU

I cicli di clock sono qui.

CPU	ISA	Microarchitettura
Intel Core i9	x86-64	Raptor Lake
AMD Ryzen 9	x86-64	Zen 4

Stessa ISA $\Rightarrow$ stesso binario su entrambe. Microarchitettura diversa $\Rightarrow$ prestazioni diverse.

I registri

Piccole celle di memoria **direttamente nel processore**. Pochissime ma enormemente piu' veloci della RAM.

64-bit	32-bit	Uso principale
`rax`	`eax`	valore di ritorno, risultati
`rbx`	`ebx`	uso generale (callee-saved)
`rcx`	`ecx`	4° argomento funzione
`rdx`	`edx`	3° argomento funzione
`rdi`	`edi`	1° argomento funzione
`rsi`	`esi`	2° argomento funzione
`rsp`	—	stack pointer (cima stack)
`rbp`	—	base pointer (base frame)
`rip`	—	instruction pointer

Stessa struttura fisica, larghezza diversa

`rax` e `eax` sono lo **stesso registro fisico**. `int` (32-bit) → `eax`. `long` (64-bit) → `rax`.

La gerarchia di memoria

La CPU puo' operare **solo su dati nei registri**. Ogni operazione segue:

$$\text{RAM} \xrightarrow{\texttt{mov}} \textbf{registro} \xrightarrow{\text{istruzione}} \textbf{registro} \xrightarrow{\texttt{mov}} \text{RAM}$$

Livello	**Latenza**	**Capacita'**
Registri	∼1 ciclo	pochi byte
Cache L1	∼4 cicli	∼64 KB
Cache L2	∼12 cicli	∼256 KB
Cache L3	∼40 cicli	∼8 MB
RAM	∼200 cicli	GB
Disco	milioni di cicli	TB

Le latenze sono proprieta' della microarchitettura, non dell'ISA.

La memoria virtuale

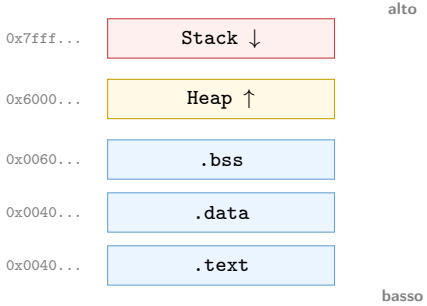
Il problema: piu' processi in esecuzione. Come si evita che si sovrascrivano?

La soluzione: ogni processo crede di avere la RAM *tutta per se'*.

La **MMU** traduce in hardware ad ogni accesso:

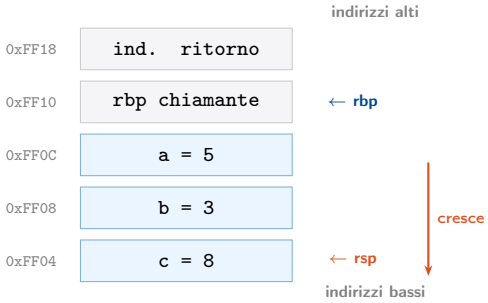
Processo	Virtuale	Fisico
A	`0x7fff1000`	`0x1FA23000`
B	`0x7fff1000`	`0x2BC10000`

Stesso indirizzo virtuale ⇒ zone fisiche **diverse e isolate**.



Lo stack — orientamento corretto

- Creato dal **SO** al lancio del processo
- Risiede nella zona ad **indirizzi alti** della memoria virtuale
- Cresce verso il basso: ogni `push` **decrementa** `rsp`
- `rsp` punta sempre all'ultima cella **occupata**



`rsp` e `rbp`

`rsp` — Stack Pointer
Punta alla cima (indirizzo piu' basso occupato). Si muove ad ogni `push`/`pop`.

`rbp` — Base Pointer
Punta alla base del frame. **Fisso** per tutta la durata della funzione.

Le istruzioni fondamentali — mov

Copia un valore da sorgente a destinazione. La sorgente rimane invariata.

```
mov eax, 5                    ; eax = 5          (immediato)
mov eax, edx                  ; eax = edx        (reg -> reg)
mov eax, DWORD PTR [rbp-4]    ; eax = memoria    (RAM -> reg)
mov DWORD PTR [rbp-4], eax    ; memoria = eax    (reg -> RAM)
```

DWORD PTR [rbp-4] — cosa significa?

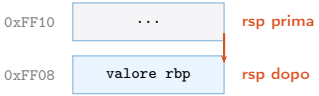
- [rbp-4] — **indirizzo** in memoria (come *p in C)
- DWORD — quanti byte leggere: 4 (come int)

BYTE PTR=1B WORD PTR=2B DWORD PTR=4B QWORD PTR=8B

Le istruzioni fondamentali — push e pop

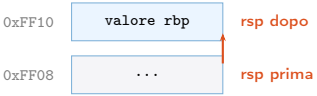
push rbp equivale a:

```
1 sub rsp, 8        ; rsp scende di 8 byte
2 mov [rsp], rbp    ; scrive in cima
```



pop rbp equivale a:

```
1 mov rbp, [rsp]    ; legge dalla cima
2 add rsp, 8        ; rsp sale di 8 byte
```



Lo stack cresce verso il basso

push ⇒ rsp **decresce** (verso indirizzi piu' bassi). pop ⇒ rsp **cresce**.

Le istruzioni fondamentali — `add`, `ret`

`add` (e aritmetica in generale)

```
add eax, edx    ; eax = eax + edx
add eax, 5      ; eax = eax + 5
sub eax, edx    ; eax = eax - edx
```

Il risultato va sempre nel **primo operando**.

`ret` — ritorno al chiamante

`call` salva l'indirizzo di ritorno in cima allo stack. `ret` lo preleva e lo mette in `rip`:

```
ret     ; equivale a: pop rip
```

`rip` = instruction pointer: la prossima istruzione da eseguire.

Prologo ed epilogo di ogni funzione

```
push rbp          ; salva rbp del chiamante
mov  rbp, rsp     ; nuova base frame
; ... corpo della funzione ...
pop  rbp          ; ripristina rbp del chiamante
ret
```

Variabili locali: dalla C all'assembly

Codice C

```c
int main() {
    int a = 5;
    int b = 3;
    int c = a + b;
    return c;
}
```

```
gcc -O0 -S -masm=intel
```

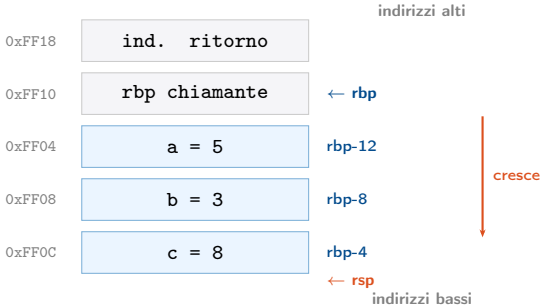
Assembly prodotto

```asm
main:
  endbr64
  push    rbp
  mov     rbp, rsp
  mov     DWORD PTR -12[rbp], 5  ; a = 5
  mov     DWORD PTR -8[rbp],  3  ; b = 3
  mov     edx, DWORD PTR -12[rbp]
  mov     eax, DWORD PTR -8[rbp]
  add     eax, edx               ; a + b
  mov     DWORD PTR -4[rbp], eax ; c
  mov     eax, DWORD PTR -4[rbp] ; ritorno
  pop     rbp
  ret
```

Variabili locali: layout sullo stack

Le variabili locali vivono a **indirizzi piu' bassi** rispetto a `rbp` (offset negativi).

Gli indirizzi **decrescono** man mano che si allocano variabili.



Perche' la CPU usa due registri?

La CPU **non puo'** sommare direttamente due locazioni di memoria. Deve passare per i registri:

$$[\texttt{rbp-12}] \xrightarrow{\texttt{mov}} \texttt{edx}$$

$$[\texttt{rbp-8}] \xrightarrow{\texttt{mov}} \texttt{eax}$$

$$\texttt{eax} \xleftarrow{\texttt{add}} \texttt{eax + edx}$$

L'effetto delle ottimizzazioni

Con `-O0` (nessuna ottimizzazione)

```
1 mov DWORD PTR -12[rbp], 5
2 mov DWORD PTR -8[rbp],  3
3 mov edx, DWORD PTR -12[rbp]
4 mov eax, DWORD PTR -8[rbp]
5 add eax, edx
6 mov DWORD PTR -4[rbp], eax
7 mov eax, DWORD PTR -4[rbp]
```

7 istruzioni, 5 accessi in memoria

Con `-O2` (ottimizzazioni attive)

```
1 mov eax, 8
2 ret
```

Il compilatore calcola 5+3=8 a **compile time** ed elimina tutto il resto!

Perche' studiare con `-O0`

Le ottimizzazioni nascondono la corrispondenza C $\leftrightarrow$ assembly. Per capire i meccanismi, usare sempre `-O0`.

Cosa sono le Calling Conventions

Quando una funzione ne chiama un'altra, serve un **accordo** su:

- dove mettere i **parametri** prima della `call`
- dove trovare il **valore di ritorno** dopo
- quali **registri** preservare e quali no
- come deve essere allineato lo **stack**

Non e' l'hardware

La CPU non conosce le calling conventions. Sono un **accordo software** tra compilatori, linker e sistema operativo. Senza di esso, codice compilato in momenti diversi non potrebbe interoperare.

Su Linux/macOS x86-64

Si usa la **System V AMD64 ABI**.

Primi 6 argomenti interi nei registri:

Argomento	Registro
1°	`rdi` / `edi`
2°	`rsi` / `esi`
3°	`rdx` / `edx`
4°	`rcx` / `ecx`
5°	`r8` / `r8d`
6°	`r9` / `r9d`
7°+	sullo stack

Ritorno: `rax` / `eax`

Calling Conventions in azione

Codice C

```c
int somma(int a, int b) {
    return a + b;
}

int main() {
    int r = somma(3, 5);
}
```

Assembly — chiamante (`main`)

```asm
mov  edi, 3       ; 1o argomento -> edi
mov  esi, 5       ; 2o argomento -> esi
call somma
; valore di ritorno ora in eax
mov  DWORD PTR -4[rbp], eax
```

Assembly — chiamato (`somma`)

```asm
somma:
  push  rbp
  mov   rbp, rsp
  mov   DWORD PTR -4[rbp], edi  ; salva a
  mov   DWORD PTR -8[rbp], esi  ; salva b
  mov   edx, DWORD PTR -4[rbp]
  mov   eax, DWORD PTR -8[rbp]
  add   eax, edx   ; eax = a + b = 8
  pop   rbp
  ret              ; ritorna, eax = 8
```

Caller-saved vs Callee-saved

I registri sono **condivisi** tra chiamante e chiamato. Chi e' responsabile di preservarli?

Caller-saved (volatili)

Il **chiamante** li salva se gli servono dopo la `call`.

Il chiamato puo' usarli liberamente.

`rax rcx rdx rsi rdi`
`r8 r9 r10 r11`

Dopo una `call`, questi valori vanno considerati **distrutti**.

Callee-saved (non volatili)

Il **chiamato** li salva se li usa e li ripristina prima di `ret`.

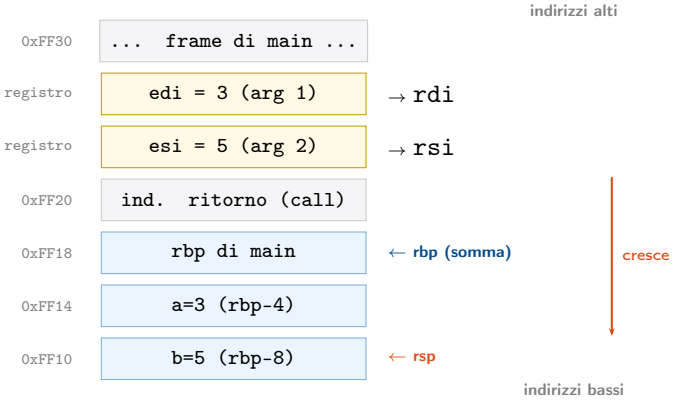
Il chiamante li trova **intatti** dopo la `call`.

`rbx rbp`
`r12 r13 r14 r15`

Usati per variabili che devono sopravvivere alle chiamate.

Il frame durante una chiamata a funzione

Stato dello stack nel momento in cui `main` chiama `somma(3,5)`:



Il Linker — Cosa fa

Il linker combina piu' file oggetto e librerie in un unico eseguibile.

1. Symbol Resolution

Ogni `.o` ha simboli **definiti** e **non definiti**. Il linker abbina ogni uso alla sua definizione.

`main.o` usa `printf` (non definito)
`libc.a` definisce `printf` ✓

Se manca: `undefined reference`

2. Merging delle sezioni

Le sezioni di tutti i `.o` vengono fuse:

- tutte le `.text` → `.text`
- tutte le `.data` → `.data`
- tutte le `.bss` → `.bss`

3. Relocation

Il compilatore usa indirizzi placeholder `0x0` per riferimenti non risolti. Il linker sostituisce tutti i placeholder con gli indirizzi reali una volta noto il layout.

Linking statico vs dinamico

Statico (`.a`)

Il codice della libreria viene **copiato** nell'eseguibile.

- Eseguibile autonomo
- Piu' pesante
- Nessuna dipendenza runtime

```
gcc -static main.c -o main
```

Dinamico (`.so`)

Solo un **riferimento** alla libreria. Il codice viene caricato a **runtime**.

- Eseguibile piu' leggero
- Libreria deve essere presente
- Aggiornamento automatico

```
ldd ./main   # mostra dipendenze
```

Il dynamic linker `ld-linux.so` risolve i riferimenti a runtime

Strumenti pratici

Ispezionare l'assembly

```
# Produrre il .s
gcc -O0 -S -masm=intel main.c

# Disassemblare un binario
objdump -d -M intel main

# Ispezionare sezioni ELF
readelf -a main

# Symbol table
nm main.o
```

Dipendenze e debug

```
# Dipendenze dinamiche
ldd ./main

# Debug con GDB
gdb ./main
(gdb) break main
(gdb) run
(gdb) info registers
(gdb) x/8x $rsp
```

Compiler Explorer: `https://godbolt.org` — C e assembly affiancati in tempo reale